

Program design and analysis

- Program-level performance analysis.
- Optimizing for:
 - Execution time.
 - Energy/power.
 - Program size.
- Program validation and testing.
- Safety and security.

CODE OPTIMIZATION

CODE OPTIMIZATION

Code optimization is: “the process of improving the efficiency, performance, and overall quality of software code without changing its functionality.”

Its main objectives are to: Reduce resource consumption. Decrease execution in time, Efficient memory utilization

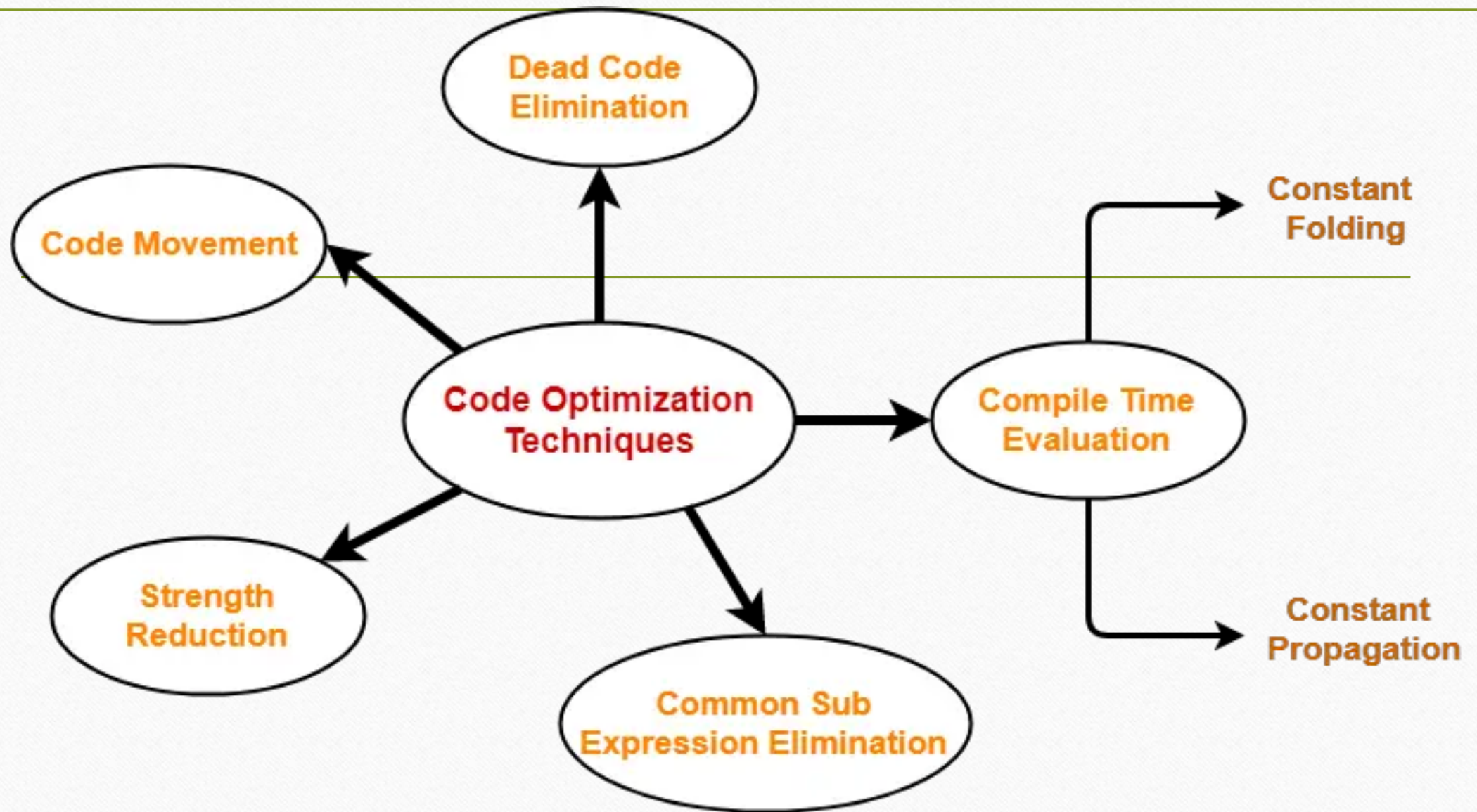
Types of Code Optimization: The optimization process can be broadly classified into two types :

1.Machine Independent Optimization: This code optimization phase attempts to improve the **intermediate code** to get a better target code as the output. The part of the intermediate code which is transformed here does not involve any CPU registers or absolute memory locations.

2.Machine Dependent Optimization: Machine-dependent optimization is done after the **target code** has been generated and when the code is transformed according to the target machine architecture. It involves CPU registers and may have absolute memory references rather than relative references. Machine-dependent optimizers put efforts to take maximum **advantage** of the memory hierarchy.

Two issues in code optimization is

1. Semantic equivalence of the source program must not be changed
2. It must be achieved without changing the algorithm of the program



A) CONSTANT FOLDING Example-

$$\text{Circumference of Circle} = (22/7) \times \text{Diameter}$$

Here,

- This technique evaluates the expression $22/7$ at compile time.
- The expression is then replaced with its result 3.14.
- This saves the time at run time.

B) Constant Propagation-

Example-

$$\text{pi} = 3.14$$

$$\text{radius} = 10$$

$$\text{Area of circle} = \text{pi} \times \text{radius} \times \text{radius}$$

This technique substitutes the value of variables 'pi' and 'radius' at compile time.

- It then evaluates the expression $3.14 \times 10 \times 10$.
- The expression is then replaced with its result 314.
- This saves the time at run time.

2. Common Sub-Expression Elimination-

Code Before Optimization

```
S1 = 4 x i  
S2 = a[S1]  
S3 = 4 x j  
S4 = 4 x i // Redundant Expression  
S5 = n  
S6 = b[S4] + S5
```

Code After Optimization

2. Common Sub-Expression Elimination-

Code Before Optimization

```
S1 = 4 x i  
S2 = a[S1]  
S3 = 4 x j  
S4 = 4 x i // Redundant Expression  
S5 = n  
S6 = b[S4] + S5
```

Code After Optimization

```
S1 = 4 x i  
S2 = a[S1]  
S3 = 4 x j  
S5 = n  
S6 = b[S1] + S5
```


3. Code Movement

Code Before Optimization

```
for ( int j = 0 ; j < n ; j ++ )  
{  
  x = y + z ;  
  a[j] = 6 x j ;  
}
```

Code After Optimization

3. Code Movement

Code Before Optimization

```
for ( int j = 0 ; j < n ; j ++ )  
{  
  x = y + z ;  
  a[j] = 6 x j ;  
}
```

Code After Optimization

```
x = y + z ;  
for ( int j = 0 ; j < n ; j ++ )  
{  
  a[j] = 6 x j ;  
}
```


4. Dead Code Elimination-

Code Before Optimization

```
i = 0 ;  
if (i == 1)  
{  
a = x + 5 ;  
}
```

Code After Optimization

4. Dead Code Elimination-

Code Before Optimization

```
i = 0 ;  
if (i == 1)  
{  
a = x + 5 ;  
}
```

Code After Optimization

```
i = 0 ;
```

5. Strength Reduction-

Code Before Optimization

$$B = A \times 2$$

Code After Optimization

Replace an expression having more priority with less priority expression

5. Strength Reduction-

Code Before Optimization

$$B = A \times 2$$

Code After Optimization

$$B = A + A$$

Loop Optimization Techniques: is the process of increasing execution speed and reducing the overheads associated with loops.

Code optimization can be done in loops

1. Code motion technique or code movement

3. Code Movement

Code Before Optimization

```
for ( int j = 0 ; j < n ; j ++ )  
{  
  x = y + z ;  
  a[j] = 6 x j ;  
}
```

Code After Optimization

```
x = y + z ;  
for ( int j = 0 ; j < n ; j ++ )  
{  
  a[j] = 6 x j ;  
}
```

We optimize the code in inner loop

<https://www.codingninjas.com/studio/library/loop-optimization>

2. Induction variable

Variable x is an induction variable of loop L such that

Value of the induction variable of loop get changed every time – either incremented or decremented

Code Motion (Frequency Reduction)

Before optimization:

```
while(i<100)
{
  a = Sin(x)/Cos(x) + i;
  i++;
}
```

After optimization:

```
t = Sin(x)/Cos(x);
while(i<100)
{
  a = t + i;
  i++;
}
```

Before optimization:

B1

i:= i+1

x:= 3+i

y:= a[x]

if y< 15, goto B2

i and x are locked,

After optimization:

B1

i:= i+1

x:= x+4

y:= a[x]

if y< 15, goto B2

if i is incremented by 1 then x is incremented by 3. So, i and x are induction variables.

<https://www.geeksforgeeks.org/loop-optimization-in-compiler-design/>

3. Loop invariant method

Computation inside the loop is avoided and thereby computation overhead on compiler is avoided

Before optimization:

```
for (int i=0; i<10;i++)  
t= i+(x/y);  
... end;
```

After optimization:

```
s = x/y;  
for (int i=0; i<10;i++)  
t= i+ s;  
... end;
```

<https://www.geeksforgeeks.org/loop-optimization-in-compiler-design/>

4. Loop unrolling

Number of jumps and tests can be reduced by writing the code two times

Before optimization:

```
#include <stdio.h>
```

```
void main()
{
    int i = 1;
    int a[100], b[100];
    while(i<100)
    {
        a[i] = b[i];
        i++;
    }
}
```

Before optimization code, a loop is running 100 times.

We can optimize this code by repeating the statements so that the loop runs only 50 times.

Hence, the **optimized code** will be-

```
#include <stdio.h>
```

```
void main() {
    int i = 1;
    int a[100], b[100];
    while(i<100) {
        a[i] = b[i];
        i++;
        a[i] = b[i];
        i++;
    }
}
```

5. Loop fusion or loop jamming

Several loops are merged to one loop

Before optimization:

```
#include <stdio.h>

void main() {
    int a[10], b[10], i;
    for(i = 0; i < 10; i++)
        a[i] = 1;
    for(i = 0; i < 10; i++)
        b[i] = 2;
}
```

After optimization:

```
#include <stdio.h>

void main() {
    int a[10], b[10], i;
    for(i = 0; i < 10; i++) {
        a[i] = 1;
        b[i] = 2;
    }
}
```

<https://www.geeksforgeeks.org/loop-optimization-in-compiler-design/>

Compiler optimizations

Function inlining replaces a subroutine call to a function with **equivalent code to the function body**. By substituting the function call's parameters into the body, the compiler can generate a copy of the code that performs the same operations but **without the subroutine overhead**. Cpp provides an inline qualifier that allows the compiler to substitute an inline version of the function. In C, programmers can perform inlining manually or by using a preprocessor macro to define the code body.

Outlining is the **opposite operation to inlining**- a set of **similar sections of code replaced with calls to an equivalent function**. Although inlining eliminates function call overhead, it also increases program size. Inlining also inhibits sharing of the function code in the cache because the inlined copies are distinct pieces of code, they cannot be represented by the same code in the cache. Outlining is sometimes useful to improve the cache behavior of common functions.

https://compileroptimizations.com/category/dead_code_elimination.htm

simple C loop: **Loop Unrolling**

```
for (i = 0; i < N; i++)  
{  
  a[i] = b[i] * c[i];  
}
```

This loop is executed a fixed number of times, namely, N. It initializes the loop variable i, updates its value on every iteration, and tests it to see whether to exit the loop. However, because the loop is executed a fixed number of times, we can generate more direct code.

If we let $N \geq 4$, then we can substitute this straight-line code for the loop:

```
a[0] = b[0] * c[0];  
a[1] = b[1] * c[1];  
a[2] = b[2] * c[2];  
a[3] = b[3] * c[3];
```

This unrolled code has no loop overhead code at all, that is, no iteration variable and no tests. But the unrolled loop has the same problems as the inlined procedure: it may interfere with the cache and expands the amount of code required.

Function Outlining

<https://developer.arm.com/documentation/ka004653/1-0>

Compiler optimizations

- Inlining
- Loop transformations- 1. loop unrolling 2. Loop fusion
- Dead code elimination
- Register allocation
- Scheduling
- Instruction selection
- Cache oriented loop optimization

https://compileroptimizations.com/category/dead_code_elimination.htm

Loop tiling is a compiler technique that divides loops into smaller blocks to improve the efficiency of memory usage

Loop tiling is a well-known compiler transformation for both sequential and parallel programs optimization.

<https://open-catalog.codee.com/Glossary/Loop-tiling/>

```
x = 500; // Declaring variable x
while(x > 1) // while condition to perform loop
{
    y = a + b;
    if (y / c == 1) // An variable is used in loop for following assignment k = k + constant.
    {
        printf("/z", k); // print output
    }
}
```

```
x = 500; // declaring variable x
y = a + b; // calculations
while(x > 1)
{
    if (y / c == 1)
    {
        printf("/z", k); // print output
    }
}
```

A **logic analyzer** is an electronic measurement device that captures and displays multiple signals from a digital design. It is an excellent tool for checking and debugging ICs, digital systems, circuits such as embedded systems, electronic control units, computers, etc. A modern logic analyzer can convert the captured data into timing diagrams, protocol decodes, state machine traces, and assembly language.

<https://evision-systems.com/what-is-a-logic-analyzer/#:~:text=A%20logic%20analyzer%20is%20an,units%2C%20computer%20peripherals%2C%20etc.>

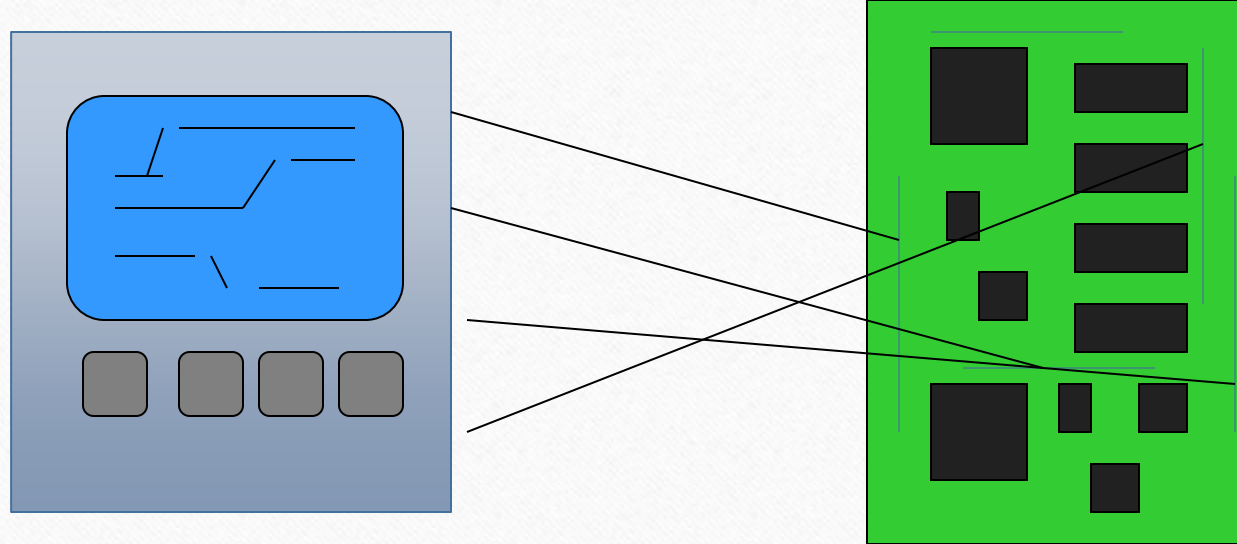
<https://www.electronicwings.com/8051/getting-started-with-8051-and-keil-ide>

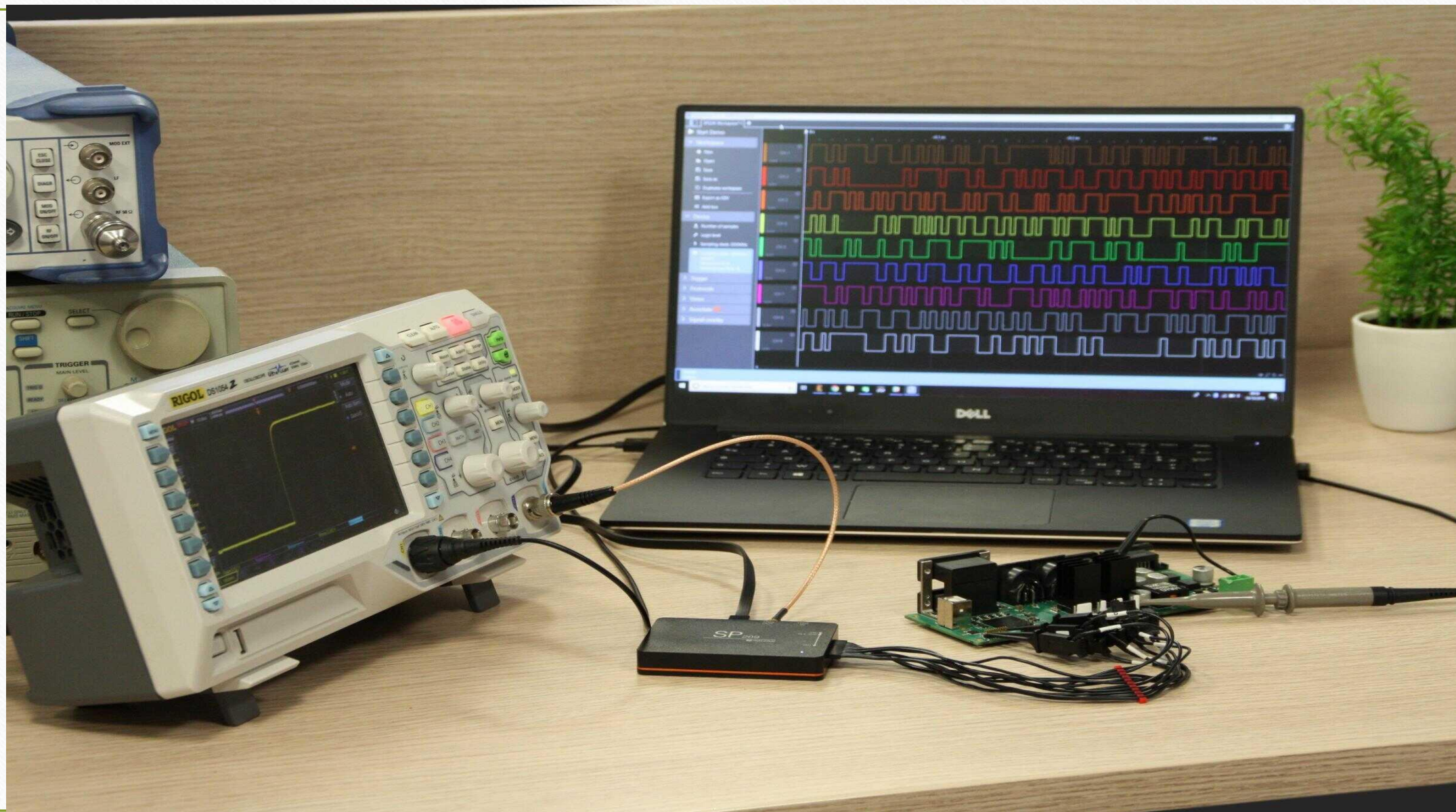
What Is Logic Analyzer

- One of the most fundamental tools used for debugging and monitoring a communication interface on an embedded system is a logic analyzer.
- A logic analyzer is a powerful device that physically interfaces to the hardware signal lines similar to how a multi-channel oscilloscope would be connected.
- Unlike the oscilloscope, a logic analyzer can't detect the voltage range but
- Instead detects whether the channel is at a digital logic 0 (ground) or logic 1 (supply).
- This helps to remove distracting features such as rise times, overshoots, noise and other signal artifacts from the signal.

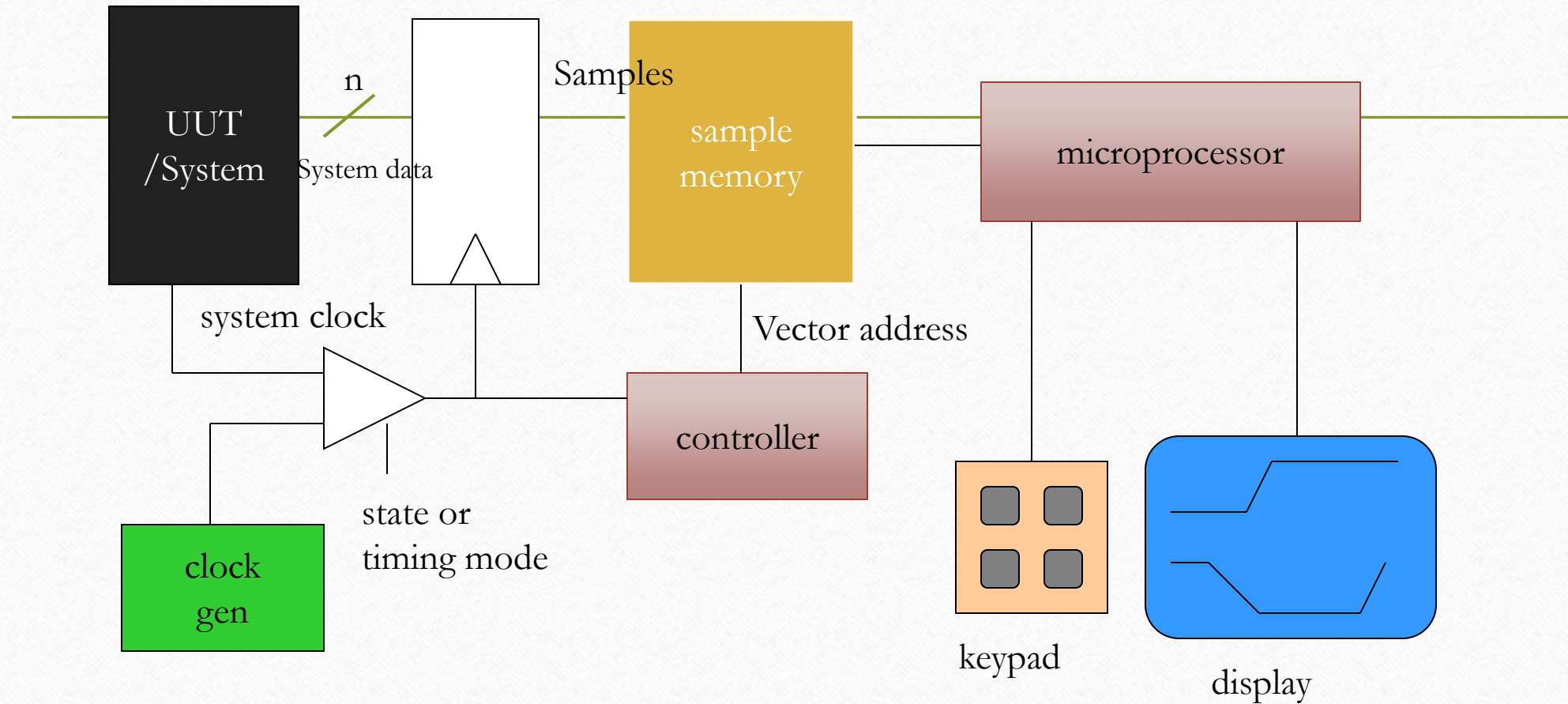
Logic analyzers

- A logic analyzer is an array of low-grade oscilloscopes:
 - Most of the logic Analyzers supports at a minimum of 8 Channels
 - O/P displayed in the form of strip chart, allows each channel is compared with all other channels at same instant of time
-





Logic analyzer architecture



The system's data signals are sampled at a latch within the logic analyzer; the latch is controlled by either the system clock or the internal logic analyzer sampling clock, depending on whether the analyzer is being used in **state or timing mode**. Each sample is copied into a vector memory under the control of a state machine. The latch, timing circuitry, sample memory, and controller must be designed to run at high speed because several samples per system clock cycle may be required in timing mode. After the sampling is complete, an embedded microprocessor takes over to control the display of the data captured in the sample memory.

Logic analyzers typically provide a number of formats for viewing data. One format is a timing diagram format. Many logic analyzers allow not only customized displays, **such as giving names to signals, but also more advanced display options. For example, an inverse assembler can be used to turn vector values into microprocessor instructions. The logic analyzer does not provide access to the internal state of the components, but it does give a very good view of the externally visible signals. That information can be used for both functional and timing debugging.**

The logic analyzer allows different types of standard interfaces to be automatically decoded

- Depending on the analyzer these can include UART, SPI, I2C, CAN and many more.
- Even if the interface that is being debugged can't automatically be decoded, the signals can at least be recorded and then manually decoded.
- Most logic analyzers have similar functions to an oscilloscope. These are functions like adjusting timing, sample rates and trigger modes.

This is done by sprinkling some test code before and after the function calls that toggle the input / output bit. This is extremely helpful when analyzing how long it takes for different tasks in the system to run. The logic analyzer can be setup to measure the width of each of the toggles.

- An average can then be taken to get the average run time.
- It is helpful to also keep the maximum and minimum run times observed.
- This can be then used to understand the timing of the entire system